Algoritmos e Lógica de Programação

1. Algoritmos

1.1 Introdução

No dia a dia, sempre que pensamos, falamos ou escrevemos, estamos necessariamente usando a **Lógica,** ou a **Ilógica** (isto ocorre quando estamos falando ou escrevendo sem pensar de maneira correta e ordenada).

Quando pensamos e falamos, a **palavra falada** representa nosso pensamento.

Quando pensamos e escrevemos, a **palavra escrita** representa nosso pensamento.

Portanto, quando queremos escrever ou falar, devemos pensar e principalmente colocar **ORDEM NO PENSAMENTO**, ou seja, **devemos usar a Lógica.**

Cada indivíduo possui sua lógica própria que pode, ou não, ser a melhor para a solução de determinado problema.

O **refinamento da lógica** de cada um depende **de** diversos fatores, entre eles:

- **Leitura constante de bons livros, principalmente aqueles que requerem o raciocínio do leitor para entender seu conteúdo (exemplos: espionagem, dramas, policial, investigação, etc.)**
- **Uma boa base matemática elementar** (lógica proposicional e conceitos de indução matemática**);**
- **Exercícios de lógicas** contidos em livros sobre programação e lógica, almanaques; e
- **Exercícios praticados** pelo indivíduo com **base em problemas do cotidiano.**

O que é lógica?

"A lógica trata da correção do pensamento."

"A lógica é a arte de pensar corretamente e, visto que a forma mais complexa do pensamento é o raciocínio, a Lógica estuda ou tem em vista a *correção do raciocínio*".

Podemos ainda dizer que a lógica tem em vista a "ordem da razão". Isto dá a entender que a nossa razão pode funcionar desordenadamente, pode pôr as coisas de pernas para o ar.

A Lógica ensina a colocar **ORDEM NO PENSAMENTO.**

Exemplo:

Todo mamífero é animal.

Todo cavalo é mamífero.

Por tanto, todo cavalo é animal. (Forbellone, 2000, p. 2André Luiz Villar)

Quando classificamos uma atitude de ILÓGICA, significa que os pensamentos não foram ordenados, antes de colocar em prática as ideias.

Exercícios de Lógica:

- **Comer uma bolacha**, considerando que.
  - O armário está fechado.
  - O pacote de bolacha está no armário.

  Nesse caso (logicamente), devo:
  - Abrir o armário.
  - Pegar o pacote de bolacha.
  - Abrir o pacote.
  - Pegar a bolacha.

Seria **ilógico** pegar o pacote de bolacha sem antes abrir o armário.

- **Assistir futebol na TV.**
  - O jogo vai ser transmitido pelo SBT.
  - A TV está desligada.

  Então (**logicamente**) devo:
  - Ligar a TV.
  - Sintonizar no SBT.
  - Assistir o futebol.

O que é lógica de programação?

Lógica de Programação é a técnica de encadear sequencias lógicas de instruções, que devem ser executadas, para cumprir determinada tarefa.

Para representar a Lógica de Programação, construímos os ALGORITMOS.

1.2 Conceitos básicos sobre Algoritmo

"Algoritmo pode ser definido como uma **sequência de passos** que visam atingir um **objetivo bem definido**." (Forbellone, 2000, p. 3).

"Algoritmo, intuitivamente, é uma **sequência finita de instruções ou operações básicas** (operações definidas sem ambiguidade e executáveis em tempo finito dispondo-se apenas de lápis e papel) cuja execução, em tempo finito, **resolve um problema computacional**, qualquer que seja sua instância." (Salvetti, 1998, p. 5).

Conforme definições acima**, devemos ter muito claro e bem definido o objetivo do nosso algoritmo**. Para tanto **devemos analisar e entender o problema proposto,** e **que resultado pretende-se** obter utilizando um algoritmo.

O algoritmo não foi criado para satisfazer às necessidades da computação. A programação de computadores é apenas um campo de aplicação dos algoritmos.

Os algoritmos são utilizados em várias áreas, tendo entre outros objetivos, a padronização do exercício de tarefas rotineiras.

Importante! – "Um algoritmo não é a solução do problema, pois, se assim fosse, cada problema teria um único algoritmo; **um algoritmo é um *caminho* para a solução de um problema**. Em geral, existem muitos (senão infinitos) caminhos que levam a uma solução satisfatória". (KOLIVER, 2009, p.11, negrito e grifo nosso).

Exemplo de algoritmo:

Problema: Trocar o pneu de um veículo. (adaptado de KOLIVER, 2009, p.11-12).

Algoritmo: **Trocar o pneu de um veículo.**

Início

Passos (Px):

P1: Abra o porta-malas do veículo.

P2: Retire o estepe, o macaco e a chave de rodas.

P3: Afrouxe os parafusos da roda.

P4: Coloque o macaco debaixo do veículo.

P5: Levante o veículo com o macaco.

P6: Retire os parafusos da roda.

P7: Retire a roda.

P8:Coloque o estepe no lugar da roda retirada.

P9: Coloque os parafusos da roda.

P10: Aperte os parafusos da roda.

P11: Abaixe o carro com o macaco.

P12: Aperte novamente os parafusos da roda.

P13: Guarde no porta-malas a roda removida, o macaco e a chave de rodas.

P14: Feche o porta-malas.

Fim.

O algoritmo acima não é a única solução para o problema **trocar pneu de um veículo**. Esse algoritmo **pode ser refinado**, por exemplo, considerando que o veículo esteja em movimento e o pneu furou. Diante disso, antes de executar o Passo 1 teríamos que fazer uma constatação, ou seja, se o veículo está em movimento. Se estiver, então deve-se estacionar o veículo no acostamento, e sinalizar o local com um triângulo.

Esse algoritmo ficaria assim:

Início

P1: Se o veículo está em movimento, então

P2: estacione o veículo.

senão

P3: Abra o porta-malas do veículo.

P4: Retire o estepe, o macaco, a chave de rodas e o triângulo.

P5: Coloque o triangulo, no acostamento, cerca de 30 metros atrás do veículo.

P6: Afrouxe os parafusos da roda.

P7: Coloque o macaco debaixo do veículo.

P8: Levante o veículo com o macaco.

P9: Retire os parafusos da roda.

P10: Retire a roda.

P11: Coloque o estepe no lugar da roda retirada.

P12: Coloque os parafusos da roda.

P13: Aperte os parafusos da roda.

P14: Abaixe o carro com o macaco.

P15: Aperte novamente os parafusos da roda.

P16: Guarde no porta-malas a roda removida, o macaco e a chave de rodas.

P17: Busque o triângulo.

P18: Guarde o triângulo no porta-malas.

P19: Feche o porta-malas.

Fim.

Outra forma de escrita do algoritmo acima está mostrada abaixo onde, considerando que um algoritmo deve ser claro, então o popularmente conhecido "macaco" para levantamento de roda, deve ser esclarecido conforme Passo 2. No passo 6 esclarece-se a forma de desapertar o parafuso da roda, e assim por diante.

Esses exemplos demonstram os diversos algoritmos que podem ser gerados a partir de um problema, sendo que cada um é passível de refinamentos, dependendo do objetivo proposto e das pessoas que irão beneficiar-se desse algoritmo.

Algoritmo: **Trocar o pneu de um veículo.**

INÍCIO

Passos:

1: Obtenha uma chave de rodas;
2: **Obtenha um dispositivo para levantamento do automóvel ("macaco") ;**
3: Obtenha uma roda substituta, com o pneu cheio;
4: Obtenha um triângulo sinalizador;
5: Posicione o triângulo sinalizador, na traseira do veículo, a uma distância de 15 metros do veículo;
6: Desaperte os parafusos da roda que está com o pneu que será trocado, no sentido ante-horário usando a chave de roda;
7: Posicione o "macaco" debaixo do veículo;
8: Posicione a borda de levantamento do "macaco", sobre trilho que esta embaixo do assoalho do veículo;
9: Acione o macaco suspendendo o veículo, até a roda não tocar mais o chão;
10: Desaperte totalmente os parafusos da roda, usando a chave de roda;
11: Retire os parafusos da roda;
12: Retire a roda;
13: Coloque a roda substituta;
14: Coloque os parafusos na roda substituta;
15: Aperte os parafusos da roda substituta;
16: Acione o macaco abaixando o veículo, até a roda tocar no chão;
17: Retire o "macaco" debaixo do veículo;
18: Reaperte bem os parafusos da roda substituta;
19: Guarde o "macaco", a chave de roda e o triângulo sinalizador;
20: Leve a roda substituída para o conserto;

FIM.

1.2.1 Características principais de um Algoritmo

✓ Um algoritmo é rigoroso

Algoritmos, mesmo inconscientemente, são comuns no nosso dia-a-dia. Por exemplo, atividades como: trocar uma lâmpada, trocar o pneu de um veículo, preparar-se para o trabalho, preparar uma refeição ou um bolo, etc., são resolvidas através de algoritmos. **Algoritmos possuem uma série de variáveis necessárias e uma sequência de diversos passos que devem ser seguidos rigorosamente** para que se consiga atingir o objetivo pré-estabelecido.

✓ Um algoritmo deve ter um padrão de comportamento

Para que a uma tarefa seja bem-sucedida, utilizando-se algoritmo, é necessário que as etapas a serem processadas sejam especificadas com clareza e exatidão, e estejam ordenadas dentro de uma sequência lógica.

Um algoritmo fixa um padrão de comportamento e as normas de execução a serem seguidas para atingir o resultado esperado, devendo ainda garantir sua **repetibilidade**, que é a garantia de que sempre que for processado, em sua forma original, irá apresentar o mesmo resultado.

✓ Um algoritmo deve terminar

Um algoritmo deve ter sempre **início e fim.**

Conforme definições acima um **algoritmo é um procedimento claro e bem definido executado passo-a-passo**, portanto, significa que **cada passo deverá ser completo antes que o próximo seja iniciado**, e ainda, **cada passo é completamente definido a partir da entrada atual e dos passos anteriores**.

Os passos devem ser definidos sem ambiguidades.

1.2.2 Desenvolvimento de Algoritmos

Conforme Salvetti (1998), **dispondo apenas de lápis e papel podemos construir algoritmos** para resolver problemas, sejam eles computacionais ou não. Entretanto, podemos usar algumas estratégias para entendimento de um problema e diminuição de sua complexidade, antes de iniciarmos a construção de um algoritmo. **As duas principais estratégias são**:

- ✓ **Técnica Top-Down**

  Essa técnica consiste em identificar partes ou etapas principais do problema apresentado (**dividir o problema**), elaborando-se um esboço de uma resolução prévia. A seguir cada etapa é analisada e detalhada sucessivamente, refinando cada uma delas (**obter coerência**), até alcançar uma sequência de operações básicas que seguidas produzam o resultado esperado.

  Se uma parte do problema não foi bem entendida aplica-se a ela novamente a técnica Top-Down.

- ✓ **Planejamento Reverso**

  Consiste em **determinar o resultado esperado** para solução do problema, ou mais especificamente, qual as saídas de dados desejadas.

  Através dessa análise, reversamente, podemos determinar as entradas (de dados) necessárias, bem como as etapas para transformação dessas entradas para obtermos as saídas determinadas e alcançarmos o resultado esperado.

  Por exemplo, supondo um problema - Cálculo do valor do ICMS de uma Nota Fiscal.

  Saída esperada: Valor do ICMS.

  Entradas (o que é necessário para obter o Valor do ICMS): Valor Total da Nota Fiscal; Alíquota do ICMS ou Sigla do Estado.

1.2.3 Dicas para desenvolvimento de Algoritmos

Não existe uma fórmula pronta para a construção de algoritmos.

A construção de algoritmos depende muito mais da lógica, criatividade, engenhosidade e experiência do construtor do que propriamente de normas ou regras que sejam estabelecidas.

Passos para construção de Algoritmos

Dado um problema para ser resolvido através de um algoritmo previamente descrito, podemos afirmar que independente de possuir ou não as propriedades acima, qualquer indivíduo pode construir um algoritmo seguindo os três passos enumerados a seguir:

1. Efetuar uma minuciosa **análise do problema;**

   Isso significa ler e entender a descrição do problema, mesmo que para isso sejam necessárias várias leituras. Nesta etapa o construtor já deve ordenar seu pensamento tendo em vista a solução do problema.

   Caso o algoritmo venha a ser escrito em uma linguagem de programação, então, o programador (construtor) deverá colocar-se na posição do computador no momento que vai simulando as instruções, para a solução do problema, em uma sequência lógica.

2. **Entendido o problema**, delinear uma estratégia para sua solução, ou seja, **construir o algoritmo.**

Tendo lido, pensado e entendido de forma clara o problema proposto, o construtor deverá nesta etapa decidir sobre a melhor estratégia para solução do problema proposto, visto que, na construção de algoritmos nem sempre teremos encontrado solução única para o problema.

No final da etapa anterior podemos ter delineado algumas opções para solução do problema, porém **nesta etapa devemos decidir e escolher a melhor proposta, e <u>usando lápis e papel</u>, construir o algoritmo para solucionar o problema proposto**.

3. **Representar esse algoritmo**, utilizando uma das formas de representação mostradas abaixo, ou seja, através de descrição narrativa, gráfica (uso de símbolos), de pseudolinguagem de programação, ou mesmo diretamente através do uso de uma linguagem de programação. Essas duas últimas formas são aplicadas para **algoritmos computacionais**.

4. **Testar exaustivamente o algoritmo.**

   Não importa se o algoritmo é computacional ou não, este deve ser testado minuciosamente, pois conforme já estudado, o algoritmo determina um padrão de comportamento, e resolve um problema qualquer que seja sua instância, portanto, testá-lo exaustivamente é uma forma de garantir uma execução, dentre outras, segura e confiável. Considerando que um algoritmo pode ser escrito usando-se apenas lápis e papel, ou seja, não necessariamente preciso de um computador, então o teste de algoritmo é conhecido como "teste de mesa", em alusão ao uso de uma mesa com lápis e papel, para sua a execução.

☞ **<u>IMPORTANTE!!!</u>Sempre que for montar um algoritmo procurar**:

✓ **Dividir o problema apresentado, nas 3 fases fundamentais de qualquer resolução lógica, ou seja:**

ENTRADA→PROCESSAMENTO→SAÍDA;

No algoritmo receita de bolo estão descritos os ingredientes necessários (**<u>entrada</u>**), uma sequência de passos ou ações a serem cumpridos (**<u>processamento</u>**), para atingir o objetivo final, que é o Bolo (**<u>saída</u>**).

Para que isso seja possível é essencial efetuar uma minuciosa **análise do problema**, buscando seu entendimento. A seguir deve-se

✓ **Descrever as instruções de forma simples e objetiva, utilizando de técnicas como:**

- **Usar só um verbo por frase;**

  Preferencialmente esse verbo deverá estar na forma imperativa (ex.: faça, calcule, use, movimente, etc.).

- **Imaginar que o algoritmo que estamos desenvolvendo será lido por uma pessoa em um local distante e, portanto, não poderá nos pedir esclarecimentos;**

  Seja claro aos escrever as instruções, não use apelidos ou palavras “inventadas”. Observe as normas cultas da linguagem utilizada.

  Se o algoritmo é computacional siga padrões para nomear algoritmos, variáveis, procedimentos, funções, etc. Use os recursos de documentação interna do programa-fonte (que representa seu algoritmo), pois é esse documento que outros programadores irão ler no momento que precisar alterar o programa.

- **Usar frases simples e curtas;**

  Lembrar sempre que um algoritmo deve constituir-se de instruções simples e objetiva.

  Algoritmo é manual de instruções e não um livro de romance.

- **Não usar palavras com sentido dúbio.**

1.2.4 Formas de representação de um Algoritmo

Um Algoritmo pode ser representado através de **frases simples** (**descrição narrativa** ou **Pseudocódigo**) ou **Símbolos** (**Fluxogramas** ou **diagrama de blocos**).

✓ **Descrição Narrativa**

É a expressão do algoritmo diretamente em linguagem natural, no nosso caso o português coloquial.

Exemplo: Trocar uma lâmpada.

Algoritmo: Trocar_Lâmpada. (Forbellone, 2000, p.4).

Início

P1: Pegue uma escada.
P2: Posicione a escada embaixo da lâmpada.
P3: Busque uma lâmpada nova.
P4: Suba na escada.
P5: Retire a lâmpada velha.
P6: Coloque a lâmpada nova.

Fim.

Esse exemplo é denominado **Algoritmo Não Computacional**, pois obviamente não pode ser transformado em um programa de computador, contendo instruções para um computador trocar uma lâmpada.

Um algoritmo é denominado **Algoritmo Computacional**, quando o mesmo pode ser executado pelo computador. Para tanto é necessário que ele seja programado utilizando-se uma linguagem de programação de computador, ou mesmo seja interpretado pelo computador através de um interpretador de linguagem. O cálculo do valor do Imposto de Renda, descontado de um funcionário é um exemplo de algoritmo computacional.

Em ambientes computacionais, a descrição narrativa é pouco usada na prática, pois o uso da linguagem natural pode ocasionar más interpretações, instruções obscuras, incompletas ou mesmo imprecisas.

Para calcular a Média Final de um aluno, em determinada disciplina, considerando duas notas informadas, é possível escrever um algoritmo na forma de Descrição Narrativa, e em seguida transformá-lo em um programa de computador, pois esse cálculo poderá ser executado por qualquer computador, e esse algoritmo é de baixa complexidade.

O uso desta forma (descrição narrativa) para **representar um algoritmo computacional, exige a definição das variáveis e constantes usadas no algoritmo** e, também, **a sua organização.**

Exemplo de um algoritmo, na forma de descrição narrativa, para calcular a média aritmética simples, de um aluno, em determinada disciplina, a partir de duas notas informadas **requer a definição das variáveis referenciadas pelo algoritmo**.

Algoritmo: Cálculo da Média em determinada disciplina.

Variáveis utilizadas:

Primeira Nota, Segunda Nota, Média Final: Tipo de Dado (ou TD) Numérico Real.

Início

P1: Solicite a Primeira nota.
P2: Obtenha a Primeira nota.
P3: Solicite a Segunda nota.
P4: Obtenha a Segunda nota.

P5: Calcule MediaFinal =(Primeira Nota + Segunda Nota) / 2.

P6: Mostre a Média Final do aluno.

Fim.

✓ Pseudocódigo

A notação de Pseudocódigo é também uma forma de representar um algoritmo, porém mais complexa e detalhada, pois **está mais próximo de uma linguagem de programação a ser utilizada** para a solução do problema através do computador.

O pseudocódigo assemelha-se bastante à forma como os programas de computador (códigos verdadeiros) são escritos, porém é um código ou linguagem falsa. Portanto o pseudocódigo. **é uma linguagem intermediária entre a nossa linguagem e a de programação**, pois quem o utiliza é norteado por um conjunto de regras, porém sem usar uma estrutura sintática tão rígida como a de uma linguagem de programação. Portanto, se pode dizer que embora não obrigatório, é relevante considerar determinada linguagem de programação, no momento de escrever um pseudocódigo.

Nesta disciplina iremos representar algoritmos através da notação de Pseudocódigo voltado para a **linguagem de programação C**.

Exemplo de um algoritmo para cálculo da média de um aluno na notação de Pseudocódigo, próximo da linguagem C.

```
// Algoritmo: Cálculo da Média de um aluno
#Incluir bibliotecas da linguagem C++
Inicio do programa principal
 // declaração de variáveis
tipo de dado Real nota1, nota2, media;
 // declaração de comandos
comando do Sistema DOS para limpar tela;          // limpar tela

pule uma linha e escreva("Informe a Nota 1: ");
leia(nota1);

pule uma linha e escreva("Informe a Nota 2: ");
leia(nota2);
   média = (nota1 + nota2) / 2;
pule uma linha e escreva("Media Final = ",media);
se (media >= 7)
        pule uma linha e escreva(APROVADO");
senão
        se(media >= 5)
           pule uma linha e escreva("Em Exame");
        senão
           pule uma linha e escreva("REPROVADO");
pule duas linhas;
pressionar enter;
Fim do Algoritmo
```

Exemplo do mesmo algoritmo acima na notação de **Pseudocódigo, próximo da linguagem de programação Pascal.**

Entradas: Notas do 1º. e 2º. bimestres - **Saída**: Média Final

```
algoritmo MEDIA;
variáveis
        Nota_1, Nota_2, Media_Final : real;
início
   limpe tela;
   Escreva('Informe a Nota 1');
   leia Nota_1;
   Escreva('Informe a Nota 1');
   leia Nota_2;
   Media_Final ←(Nota_1 + Nota_2) / 2;
   se Media_Final  >=7 então
        imprima "APROVADO"
   senão
        se Media_Final >= 5 então
             imprima "EXAME"
        senão
             imprima "REPROVADO";
  pressione uma tecla.
fim.
```

- ✓ **Símbolos (Fluxograma ou Diagrama de Blocos)**

  É uma representação gráfica dos algoritmos, onde formas geométricas diferentes representam ações distintas (entradas, instruções, comandos, saídas, etc.).

  O Fluxograma também referenciado como Diagrama de Blocos, facilita o entendimento das ideias contidas nos algoritmos, e sua transformação em comandos de linguagem de programação, ou mesmo em pseudocódigos.

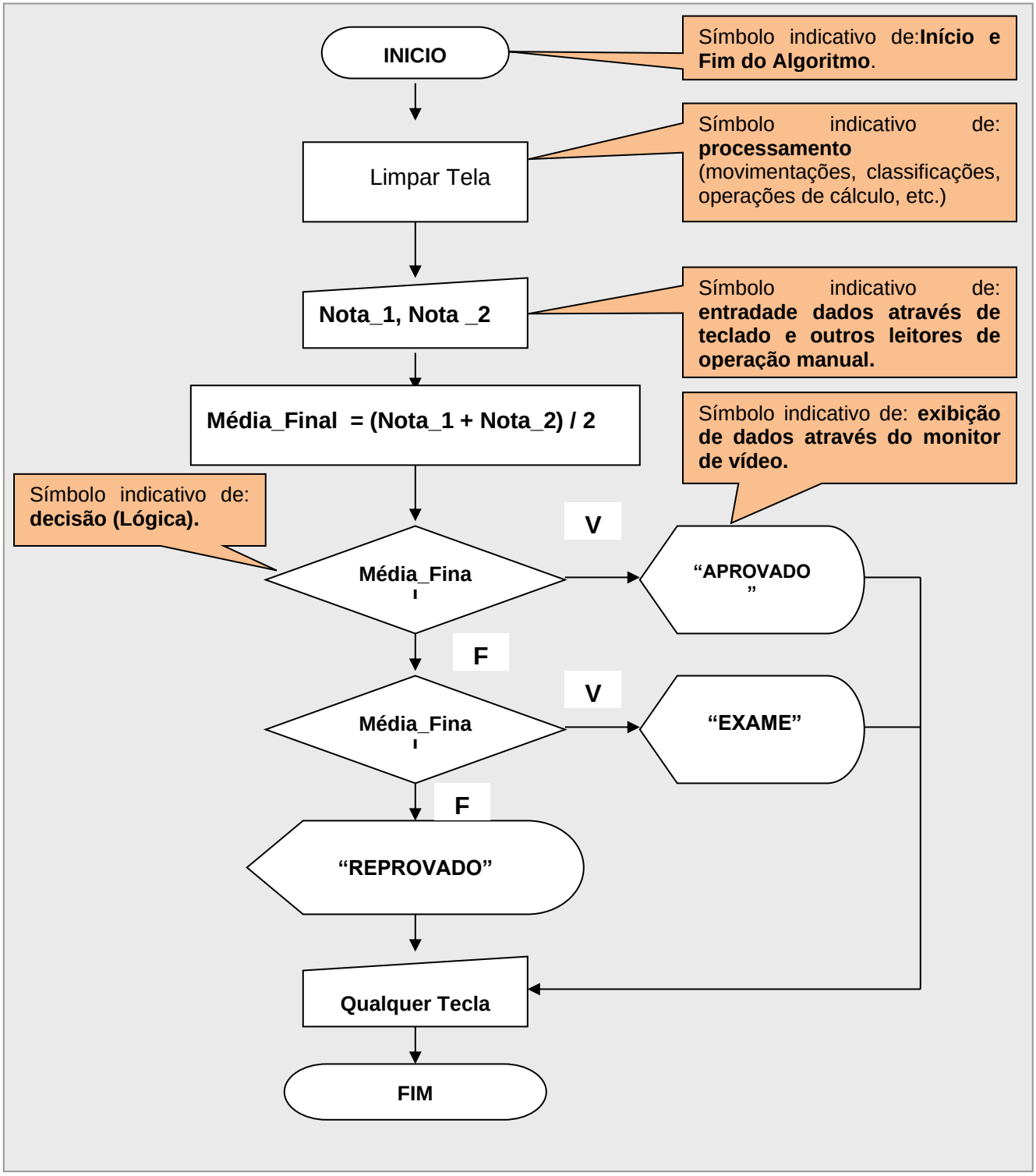
  Genericamente falando, um fluxograma resume-se a um único símbolo inicial, por onde a execução do algoritmo começa, um ou mais símbolos representando processamentos e um símbolo final, que é o ponto onde a execução do algoritmo se encerra.

  A partir do símbolo início, há sempre um único caminho orientado a ser seguido, exceto para o símbolo final, onde não existe nenhum fluxo saindo, e para o símbolo de decisão, de onde pode haver mais de um caminho de saída.

  Os primeiros fluxogramas eram construídos por programadores utilizando-se uma régua (ou gabarito) contendo a simbologia básica dos fluxogramas. Atualmente os editores de textos como

Microsoft® Word, Adobe Acrobat®, e softwares como o Microsoft® Visio, possuem a simbologia para construção de Fluxogramas, contidas em opções de Formas de representação.

Abaixo, representação simbólica (Fluxograma) do algoritmo anterior em Pseudocódigo Pascal.



✓ Programa de Computador

É a representação de um algoritmo, através de uma linguagem de programação, onde ações distintas (entradas, instruções ou comandos, saídas etc.), são representadas conforme regras sintáticas da linguagem.

Um programa escrito por um programador de computador, usando uma linguagem de programação de seu domínio, para ser executado pelo computador, precisa ser traduzido para a linguagem de máquina, que é a linguagem que o computador entende.

O processo de transformar um programa de computador escrito por um programador, em um programa em linguagem de máquina, é conhecido como compilação do programa.

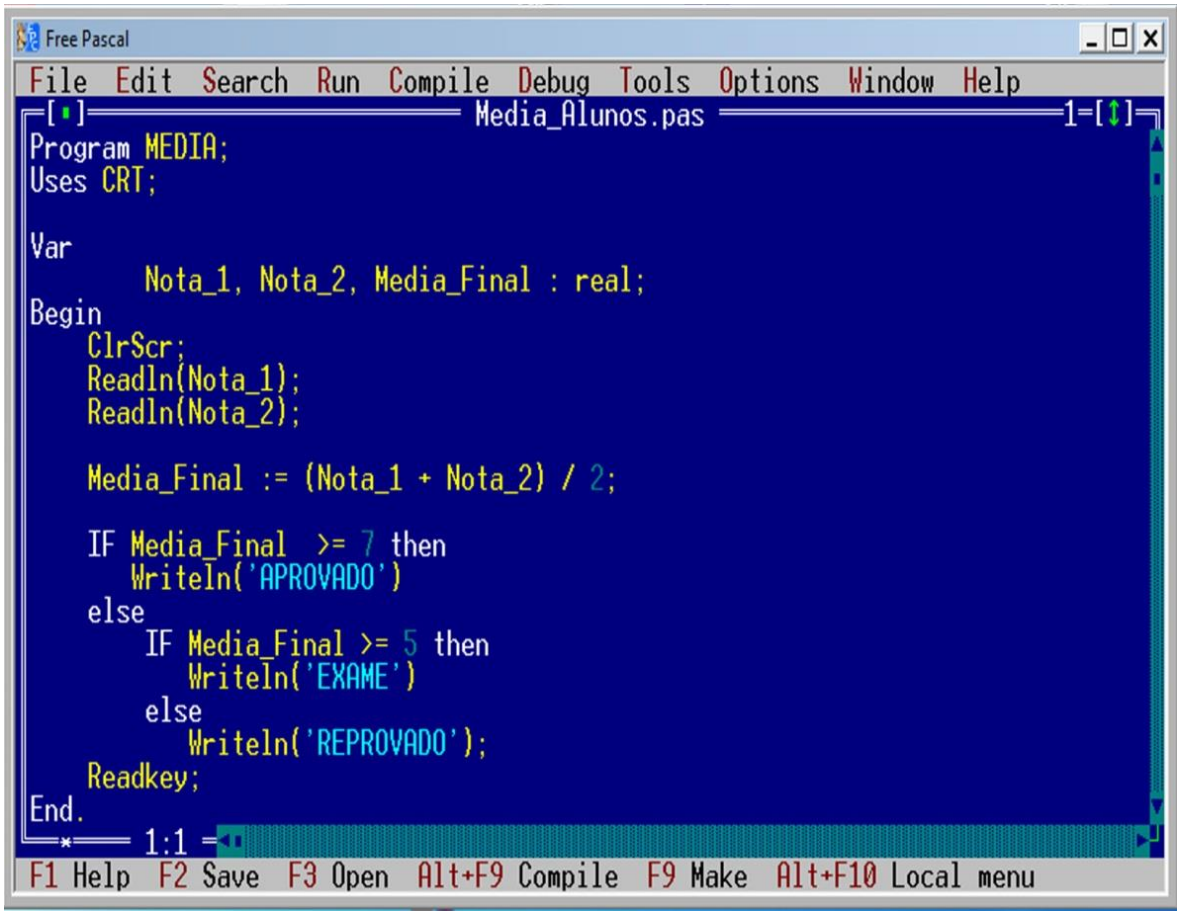
O processo de compilação será detalhado na aula sobre o ambiente de programação C++, que é a linguagem que será utilizada nesta disciplina, para validação dos algoritmos exemplos e dos exercícios propostos nas aulas.

Exemplos de Programa de Computador:

1. O mesmo algoritmo, anteriormente representado (cálculo da média de um aluno), agora está **escrito conforme regras sintáticas da linguagem Pascal**.

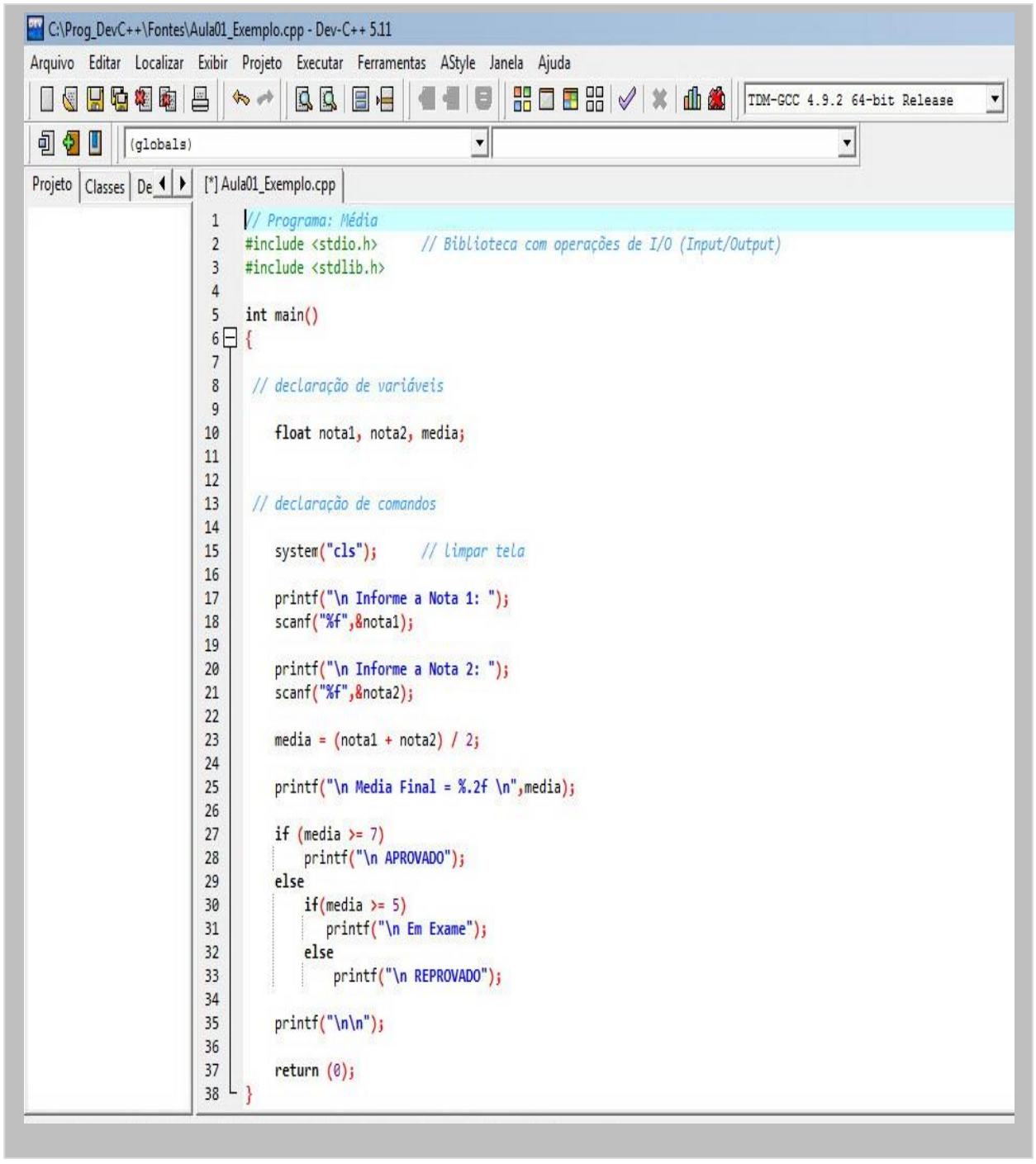
   A tela abaixo é o ambiente integrado de desenvolvimento de programas (IDE – *Integrated Development Environment*) em *Free Pascal*. Dentro desse ambiente está aberto o arquivo **Media_Alunos.pas**, que contém os comandos do programa para cálculo da média dos alunos.

   ☞ Os arquivos que contém programas escritos em determinada linguagem de programação são denominados **arquivos-fontes** ou **programas-fontes**.



2. O mesmo algoritmo, anteriormente representado (cálculo da média de um aluno), agoraestá **escrito conforme regras sintáticas da linguagem C++**.

A tela abaixo é o ambiente integrado de desenvolvimento de programas em C++ (Dev-C++). Dentro desse ambiente está aberto o programa-fonte em C++: **Aula01_Exemplo.cpp**, que contém os comandos do programa para cálculo da média dos alunos.



Importante!

Observar que na construção de algoritmos computacionais, tanto na forma de descrição narrativa quanto na forma de Fluxograma, não é dada importância à definição de variáveis, ou seja, não aparecem variáveis definidas. Já nas formas de Pseudocódigo ou Programa de computador, a definição de variáveis é obrigatória. Entretanto, o desenvolvedor do algoritmo poderá, utilizando as duas primeiras formar citadas no início do parágrafo, definir as variáveis que serão utilizadas no algoritmo.

Todavia, nesta disciplina, <u>será obrigatório definir variáveis</u> sempre que for desenvolver <u>algoritmo computacional</u>, aplicando qualquer uma das formas de representação dele.

1.2.5 Exemplo de refinamento de um algoritmo

? Problema: Calcular o tempo de casado (em anos) de uma pessoa.

✍ Pensando a Solução: Basta considerar o ano atual e o ano em que a pessoa se casou. Subtraindo o ano do casamento do ano atual obtém-se o tempo de casado.

Representação dessa solução em linguagem algorítmica:

✍ **Descrição Narrativa**

Variáveis utilizadas:

Primeira Ano Casamento, Ano Atual, Tempo Casado: TD Numérico Inteiro sem sinal.

Início

P1: Solicite o Ano Casamento.
P2: Obtenha o Ano Casamento.
P3: Considere o Ano Atual.
P4: Calcule Tempo Casado = Ano Atual - Ano Casamento.
P2: Mostre o Tempo Casado.

Fim.

✍ **Linguagem de programação Dev-C++**

A escrita do algoritmo anterior, agora usando a linguagem Dev-C++, levará a dúvida sobre como programar o passo 3 (P3), ou seja, como o computador irá considerar o Ano Atual?

Uma primeira solução será solicitar/receber o Ano Atual, conforme mostrado no **exemplo de programa 1** (Algoritmo) imediatamente abaixo.

Todavia, cabe observar que caso seja informado Ano Atual menor que o ano realmente atual, então o cálculo do tempo de casado irá retornar um número negativo. Portanto, neste caso podemos, através de comandos que serão aprendidos em outras aulas, validar a data atual informada, aceitando somente um valor que seja igual ou maior o ano corrente.

Entretanto, mesmo fazendo a validação do ano atual informado, o usuário poderá entrar com um ano que não condiz com a realidade e, nesse caso, o resultado mostrado não será confiável.

Para resolver essa situação devemos, através de procedimentos e funções dentro do programa, deixar que o próprio computador obtenha o Ano Atual, que está na sua memória (BIOS – Basic Input/Output System). O **exemplo de programa 2** mostra essa solução.

Exemplo de programa 1:

```
// Programa: Cálculo do Tempo de Casado (em anos) de uma pessoa.

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>


intmain()
{                                                   // início do programa
setlocale(LC_ALL,"Portuguese");
// Declaração de Variáveis
intano_casamento, ano_atual, tempo_casado;
// Declaração de comandos (instruções)
printf("\n\n Qual o Ano do seu casamento?  ");
scanf("%d",&ano_casamento);
printf("\n Qual o Ano Atual?  ");
scanf("%d",&ano_atual);
tempo_casado = ano_atual - ano_casamento;
printf("\n\n Você está casado há: %d anos \n\n",tempo_casado);
return 0;
}                                                   // Fim do programa
```

Exemplo de programa 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <time.h>
intmain()
{
setlocale(LC_ALL,"Portuguese");
intano_casamento, ano_atual, tempo_casado;
printf("\n\n Qual o Ano do seu casamento?  ");
scanf("%d",&ano_casamento);

// obter a data do sistema (BIOS)
char data_Str[8];          // Variável para armazenar a data no formato MM/DD/AA
strdate(data_Str);         // Procedimento que retorna data do sistema no formato MM/DD/AA
ano_atual = atoi(data_Str+6);       // função que converte dado tipo string para tipo inteiro
// observar que o computador irá converter o valor que está na posição 6 da stringdata_Str
ano_atual = ano_atual + 2000;       // Soma 2000 pois o ano retornará com duas posições

tempo_casado = ano_atual - ano_casamento;
printf("\n\n Você está casado há: %d anos \n\n",tempo_casado);
return 0;
}
```

Observar que para obter a data do sistema (BIOS) foi necessário escrever quatro linhas de instruções, além de ter conhecimentos avançados de programação, além disso foi necessário um artifício que foi a soma de 2000, no conteúdo retornado pelo computador, ou seja, retornou ano_atual = 18, então para transformar em 2018 foi necessário somar 2000.

Sabe-se que o ano atual está armazenado na BIOS com quatro posição, então para tornar o programa mais refinado o correto é utilizar uma função que retorne o ano atual com quatro posições numéricas.

☺☞ **Para essa implementação fica a sugestão de pesquisa sobre como fazer isso.**

Exercícios:

1. Elabore um algoritmo, na forma de descrição narrativa, para indicar a descrição da doença e também o medicamento que é necessário ser ministrado conforme o código da doença informado como entrada, e de acordo com a seguinte tabela:

Código da doença	Descrição da doença	Medicamento
0	Gripe	Vitamina C
1	Torcicolo	Relaxante muscular
2	Dor de cabeça	Analgésico
Outro valor qualquer	Indeterminado	Indeterminado

2. Elabore um algoritmo para resolver o seguinte problema: três homens, dois magros e um gordo, precisam atravessar um rio com um barco. O barco suporta apenas os dois homens magros ou o gordo sozinho.

3. Um homem precisa atravessar um rio com um barco que possui capacidade para carregar apenas ele e mais uma de suas três cargas, que são: um lobo, um bode e um fardo de alfafa. Porém, o lobo não pode ficar sozinho com o bode, na margem do rio, nem o bode pode ficar sozinho com o fardo de alfafa. Elabore um algoritmo que descreva como o homem pode atravessar o rio com sua carga.

4. Elabore um algoritmo na forma de Fluxograma, para calcular os juros de uma aplicação em caderneta de poupança, e mostrar o saldo atual dessa poupança.

5. Elabore um algoritmo, na forma de descrição narrativa, para calcular e mostrar o peso ideal de uma pessoa do sexo masculino, tendo como dados de entrada: a Altura e o Peso Atual da pessoa.

   A fórmula para cálculo do peso, para Homens, é:

   Peso Ideal = (72.7 x Altura) – 58

   Comparar o Peso Ideal calculado, com o Peso Atual informado, e verificar:

   Se o peso atual da pessoa estiver acima do Peso Ideal, escrever a mensagem:

   “Regime Obrigatório Já”;

   Caso contrário escrever a mensagem:

   “Você está com o peso ideal”.

Bibliografia:

- PIVA JR, D.; ENGELBRECHT, A. M.; NAKAMITI, G. S.; BIANCHI, F. **Algoritmos e Programação de Computadores**. Ed. Campus-Elsevier, 2012.
- SALVETTI, D. D.; BARBOSA, L.M. **Algoritmos**.Ed. Makron Books, 1998.
- FORBELLONE, André L. V.; EBERSPACHER, Henri Frederico. **Lógica de Programação: A construção de Algoritmos e Estrutura de Dados**. Ed. Makron Books, 2a. edição, revisada e ampliada, 2000.
- Professora Luciana Martinez**. Apostila deAlgoritmos**. IMAPES, 2001.
- KOLIVER, C.*et al.* **Introdução à construção de Algoritmos: notas de aulas.** Caxias do Sul, RS: EDUCS, 2009.